p0860R1
Atomic Access Property for mdspan

Published Proposal, 6 May 2018

This version:
https://github.com/ORNL/cpp-proposals-pub/blob/master/P0860/P0860r1.html

Authors:
Dan Sunderland
Christian Trott
H. Carter Edwards

Audience:
SG1, LEWG

Toggle Diffs:
☐ Hide deleted text

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Abstract

Extension to allow atomic operations on member of a mdspan

Table of Contents

1 **Revision History**

2 **Overview**

3 **Proposal**

References

Informative References

§ 1. Revision History

[P0860r1]

- Convert to bikeshed
- Remove `span`
- Conform to `mdspan` accessor concept

[P0860r0]

- 2017-11-08 Albuquerque LEWG feedback for P0009, P0019, and P0564 Extract atomic array reference from P0019 and create a separate paper

§ 2. Overview

High performance computing (HPC) applications use very large arrays. Computations with these arrays typically have distinct phases that allocate and initialize members of the array, update members of the array, and read members of the array. Parallel algorithms for initialization (e.g., zero fill) have non-conflicting access when assigning member values. Parallel algorithms for updates have conflicting access to members which must be guarded by atomic operations. Parallel algorithms with read-only access require best-performing streaming read access, random read access, vectorization, or other guaranteed non-conflicting HPC pattern.

An `atomic_ref` [P0019r7] is used to perform atomic operations on the non-atomic members of the referenced array. Construction of an `atomic_ref` for a non-atomic object requires the non-atomic object satisfy several conditions.

We propose the `atomic_accessor` accessor for `mdspan` [P0009r6] such that all references to elements are `atomic_ref`.

§ 3. Proposal

Add the following Accessor Policy to section 3.7 of [P0009r6]

```
struct atomic_accessor
{
  template <typename T>
  struct accessor
  {
    using pointer      = T*;
    using reference    = atomic_ref<T>;

    constexpr reference operator()(pointer p, ptrdiff_t i) const noexcept;
  };
};
```

The template argument for T shall be trivially copyable [basic.types].

constexpr reference operator()(pointer p, ptrdiff_t i) const noexcept ;

Requires: `p[i]` is aligned to the `required_alignment` of `atomic_ref` [atomic.ref.generic]

Effects: Equivalent to `return atomic_ref<T>(p[i]);`

§ References

§ Informative References

[P0860r0]

H. Carter Edwards, Christian Trott, Daniel Sunderland. Atomic Access Property for span and mdspan. URL: https://wg21.link/p0860r0